Rethinking Zero-Copy Networking with MAIO

Alex Markuze
VMware

Igor Golikov
VMware

Chen Dar
VMware

ABSTRACT

Berkeley Sockets (a.k.a, BSD, POSIX sockets) are ubiquitously used for network communication. BSD Sockets have been the *de-facto* standard API for network I/O since they were introduced almost four decades ago.

With the advent of high-speed Ethernet, the performance overhead of BSD Sockets became evident. Attempts to avoid these overheads have spurred a trend kernel bypass techniques, e.g., DPDK, AF_XDP, Netmap. By bypassing the kernel, these methods attempt to avoid the performance penalties associated with BSD Sockets, i.e., memory copy, system calls, and a slow network stack. However, with great performance comes the great responsibility of re-creating the same network infrastructure that already exists inside the kernel. Kernel developers attempt to close the performance gap by adding new capabilities, most notably XDP, MSG_ZEROCOPY, and tcp_mmap, but none of the proposed solutions is a panacea.

In this work, we propose a new paradigm for userspace networking, aiming to shrink the performance gap between BSD Sockets and kernel bypass techniques, allowing application developers to keep the standard BSD Sockets API, network stack (e.g., TCP) and network tools without compromising on performance. We introduce MAIO, a dedicated memory allocator for networking that inherently facilitates zero-copy I/O operations. We modify the kernel memory management system to implement dynamic memory segregation, introducing *I/O only pages* that are shared between the user, the kernel, and the device. These I/O pages are used only for I/O and can never be used by the kernel for any other purposes. This scheme facilitates zero-copy I/O while isolating kernel memory from the user. Additionally, we leverage existing HW capabilities (e.g., NVIDIA QPs, Intel ADQ) to facilitate isolation between processes.

MAIO is the first design to provide zero-copy networking while still taking advantage of the robust kernel network stack without compromising the system's security. It is currently used to facilitate efficient networking for our companies next-generation SD-WAN gateways.

1 INTRODUCTION

BSD Sockets provide a convenient API for user-space I/O, but network speeds have outstripped those of the CPU and memory in the past decades. Furthermore, while CPU and memory clock speeds have stagnated in the past decade, Ethernet speeds are growing steadily [1]. Modern systems need multiple CPU cores to saturate a +10GB/s network link. The most significant performance overhead is memory copying, which can take a big part of the CPU cycles [8]. Also, heavy copy operations can hurt other processes by "polluting" the shared L3 cache and putting additional pressure on the memory channels [26].

Attempts to ameliorate these data moving costs have spawned numerous partial optimizations in the kernel (e.g., splice, `sednfile`, MSG_ZEROCOPY, tcp_mmap). However, A comprehensive solution is still missing, resulting in the adoption of kernel bypass technologies (e.g., KTCP, Netmap, XDP). Kernel bypass solutions eschew the rich networking infrastructure, developed and perfected in the kernel, leaving the developers with the need to re-develop existing infrastructure (e.g., ARP, IPv4, TCP, ICMP, IGMP). In this work, we add our solution to the myriad of zero-copy solutions proposed in the past 40 years, none providing all the needed elements, .i.e, a simple and generic API coupled with low overheads.

The most significant hurdle for zero-copy networking is isolating processes form each other and from kernel memory. Unlike storage devices that enjoy zero-copy I/O support for many years, zero-copy for networking is a difficult problem. Due to the effectively random arrival pattern of network packets, it is impossible to know in advance which packet will arrive next and to what process it is intended. Due to the random arrival pattern, user memory could not be used in the receive flow. By contrast, in storage, I/O operations are issued by a local process, which provides the corresponding memory for that operation. Using process memory for I/O avoids any security issues as all I/O operations are performed without going through kernel memory. We contend that the optimal solution has been overlooked due to concerns over memory isolation. With the advent of multi-queue NICs and large memory systems, these concerns are easily addressable. In this work, we show that existing network interface cards (NICs) already have the capability to facilitate the needed memory isolation. We classify the existing solutions into five main categories(Sec .2) and show that the statically mapped shared buffer approach has remained uncharted. With most recent techniques (e.g., [8, 11, 32, 33, 37]) focusing instead on dynamic remapping and kernel bypass technologies.

We assert that the secure and performant solution for networking is a statically mapped memory region dedicated for network I/O. We demonstrate how a Memory Allocator for I/O (MAIO) approach provides generic zero-copy support for BSD Sockets without compromising the security of the system. MAIO consists of a modified Linux kernel 5.4, in which we add I/O pages to the kernels memory management system. These I/O pages are solely used for network I/O, thus insulating I/O memory, accessible to the user from the rest of the kernel's memory. Additionally MAIO has a user-space library for managing I/O memory.

To provide a fully overhead free I/O, we also work to eliminate the cost of system calls. In this case, we adopt the kernel thread approach proposed by FlexSC[34] and exemplified in `io_uring`[2]. We show that shared memory buffers coupled with kernel threads provide a holistic solution for high-performance networking.

MAIO is potentially beneficial for many VMware products like NSX and Velocloud. Additionally, MAIO is also extendable to ESX guest networking, potentially eliminating the performance gap between SR-IOV NICs and VMXNET3 para-virtual devices.

0.) System	zero-copy	exceptionless	Statically mapped	Network Stack	Generic use	comments
1.) Naive	✗	✗	✓	✓	✓	standard BSD Sockets
2.) AIO/io_submit [7]	✗	✗	✓	✓	✗	
3.) io_uring [2]	✗	✓	✓	✓	✓	
4.) splice[23]	✓	✗	✓	✓	✗	Only between kernel buffers
5.) sendfile[22]	✓	✗	✓	✓	✗	File only, no RX
6.) vmsplice[24]	✓	✗	✗	✓	✗	No completion notification
7.) MSG_ZEROCOPY[8]	✓	✗	✗	✓	✓	
8.) tcp_mmap[12]	✓	✗	✗	✓	✓	4K MTU only
9.) SOCKMAP[14]	✓	✓	✓	✓	✗	Splicing Only, eBPF
10.) XDP[32]	✓	✓	✓	✗	✗	eBPF callbacks
11.) XSK/AF_XDP [4]	✓	✗	✓	✗	✓	Kernel bypass
12.) NetMap [33]	✓	✗	✓	✗	✓	Kernel bypass
13.) DPDK [11]	✓	✓	✓	✗	✓	Kernel bypass
14.) RDMA	✓	✓	✓	N/A	✓	Specialized HW
15.) MAIO	✓	✓	✓	✓	✓	

Table 1: Network I/O solutions

2 BACKGROUND

The quest for performant and generic zero-copy techniques has yielded multiple solutions over the years; we will discuss the solutions used today, i.e., are novel or have passed the test of time. We compare the covered solutions in table 1.The Naive solution refers to standard `send_msg/recv_msg`. In the table we show weather each approach provides zero-copy and/or avoids system calls(exceptionless).

We classify the existing zero copy techniques into five categories:

(1) Dynamic remapping (e.g., tcp_mmap, MSG_ZEROCOPY[8], mikelangelo-project[37]).
(2) Kernel Bypass (e.g., DPDK, Netmap[33]).
(3) Special/Limited use-case (splice, sendfile).
(4) Specialized Hardware (e.g., RDMA).
(5) Static mapping (e.g, INSTANCE [16]).

Dynamic remapping solutions pin user data-pages to memory as they are being sent and unpinned when done. Unfortunately, this comes with a performance cost; thus, we mark dynamic remapping as a disadvantage in the table. Kernel bypass techniques eschew the kernel network stack support, standard tools, and stats (e.g.,`ifconfig, ethtool, ip`). Special purpose commands can not support generic networking and thus marked with a disadvantage under the generic use column in table 1.

2.1 Dynamic Remapping

Several system calls exist that allow zero-copy between user-space and the kernel e.g., vmemsplice[24],MSG_ZEROCOPY[8] and tcp_mmap[12]. Zero-Copy is enabled by temporarily mapping user-space pages to the kernel. These solutions work well when the sent/received data is large; for smaller packets, dynamic mapping costs become evident. We have found that even with 16KB sends; the performance costs are high. Besides, these system calls come with additional limitations. `vmemsplice`, does not have completion notifications. As a consequence, the user does not know when it is safe to modify the data, which may result in data corruption. Tcp_mmap, on the other hand, works only in full-page granularity, resulting in a limited use case.

2.2 Kernel Bypass

The performance costs of BSD Sockets have led some developers to the notion that the existing networking ecosystem is broken and gave rise to new holistic designs. We will shortly cover the three most prominent examples Intel's DPDK [11], AF_XDP [4] and Netmap [33].

NetMap. A new ecosystem with its own API. NetMap[33] uses shared memory buffers and modified device drivers that utilize dedicated rings to send and receive NetMap buffers. NetMap facilitates batched I/O in order to mitigate system call costs.

XSK/AF_XDP. XDP Sockets (XSK), are a similar concept to Netmap implemented by extending the XDP [32] infrastructure to bypass the kernel network stack.

DPDK. The Data Plane Development Kit (DPDK)[11] provides a set of libraries for the fast processing of network packets, including cryptographic libraries and memory management. DPDK includes a full re-implementation of many device drivers in user-space.

These solutions provide efficient zero-copy I/O but, in doing so, lose the stable and well-maintained kernel network stack. Successful utilization of such designs demands a high level of expertise in these specific systems and the need to build a custom network infrastructure.

2.3 Special Use System calls.

Several solutions were created to eliminate costly data movement in specific scenarios. We will discuss existing APIs that facilitate zero-copy I/O.

splice/sosplice[23, 29]. It is a non-standard POSIX API with different names on different operating systems. This system call is used to move data between two file descriptors[1], without copying.
sendfile[22]. Similarly to splice, sendfile is a system call used for moving data between two file descriptors without copying. In this case, the source must be an actual file, i.e., must support mmap; namely, it cannot be a socket.

2.4 eBPF

Recently eBPF[36], an in-kernel virtual machine was introduced into the Linux kernel. eBPF allows running user programs inside the kernel. Multiple projects[15] were created upon the eBPF infrastructure; some also enhance networking. We compare the performance of applicable projects to MAIO in Sec. 5.
XDP[32]. Intended for high-speed network processing, at the earliest stage possible, before any meta-data is created. XDP is used for fast packet processing e.g., DDOS mitigation, load balancing, providing an alternative to kernel bypass techniques.
SOCKMAP[14]. Enables, secure redirection at the socket layer, also can be used for socket splicing[20]. SOCKMAP allows to execute user logic and perform zero-copy I/O without additional overheads.

2.5 Remote Direct Memory Access

Remote Direct Memory Access (RDMA), is a family of protocols (e.g., Infiniband[30], iWarp[6], ROCE[38], Omnipath[3]), dating back 30 years. RDMA allows for direct access, i.e., read and write to the memory of a remote machine. RDMA facilitates zero-copy I/O, in fact, I/O without any CPU involvement. The downside of RDMA is that specialized hardware (e.g., NICs, routers, switches) is needed, and RDMA applications cannot communicate directly with Ethernet-based applications. RDMA is mostly found in HPC environments[39].

2.6 Async I/O API

The Linux kernel has a couple of libraries for batching I/O operations asynchronously; this allows the application to delegate the actual I/O work to a kernel thread and free-up the application core. However, neither provide support for zero-copy operations in networking, which accounts for the highest performance toll.
AIO/`io_submit` [7], initially added to support disk I/O, can also be utilized for networking and socket splicing[21]. While not a zero-copy solution, AIO allows to batch system calls and thus ameliorate the cost of system calls.
`io_uring` [2], a newer alternative to AIO, providing infrastructure for issuing batched I/O asynchronously. `io_uring` utilizes kernel threads to perform the actual data copy and I/O operation. In our own work on MAIO, we adopted a similar approach of using a kernel thread to poll a user ring in order to avoid the cost of system calls. But not the high costs of copying data with network I/O.

3 DESIGN AND IMPLEMENTATION

Our main goal when designing MAIO was to preserve a standard BSD Sockets API while providing a seamless zero-copy I/O support. A secondary goal is to eliminate costly system calls from the data path. However, we also needed to make sure that the security of the system is preserved.

We propose MAIO, a Memory Allocator for I/O, used exclusively by a specific user-process and the kernel device drivers. We adopt the allocation mechanics proposed in DAMN[27]. I.e., the allocator is based on two known mechanisms; a page_frag mechanism [13] over 64KB buffers, these buffers in turn allotted by a magazine allocator [5]. This scheme allows for the efficient allocation of variable size buffers in the kernel. We mark the metadata of these MAIO pages to indicate that they belong to MAIO. With this tag, anytime an I/O memory page is freed in the kernel (e.g., on TX completion), it will return to MAIO rather than to the kernel page allocator.

To facilitate zero-copy in user-space, MAIO pages are mapped *once* to the privileged user-space process's address space. The user-space then allocates from the kernel a batch of buffers for its use (i.e., zero-copy send). Another way the user-space process can get MAIO buffers is by performing zero-copy receive. The user-space process can return memory to the kernel via a exception-less mechanism described in Sec. 3.1. The MAIO buffers are depicted in Fig. 1 as dashed boxes, same pages are used in TX and RX by the user (marked **[a]** and **[b]**) and TX/RX by the device driver (marked **[c]** and **[d]**). With zero-copy operations, the page's ownership is transferred between the user and kernel on each I/O operation or by explicitly allocating/freeing pages from/to the kernel.

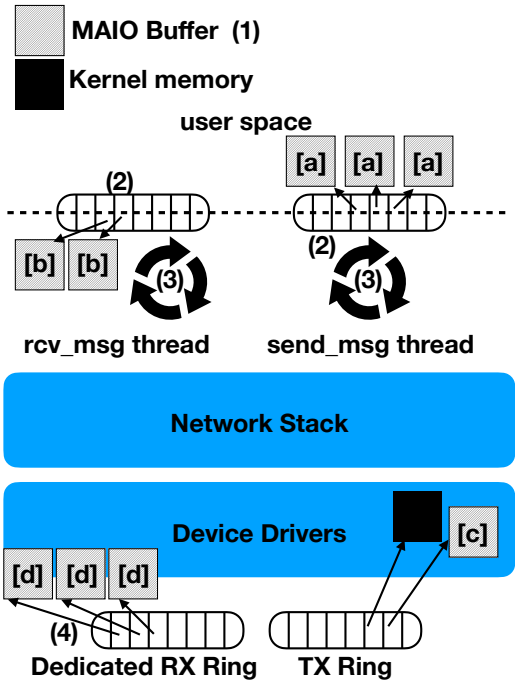


Figure 1: 1. MAIO shared memory buffers 2. shared io rings for exception-less system calls 3. A kernel thread executing I/O operations 4. A dedicated RX ring. [a] zero-copy send_msg [b] zero-copy recvmsg [c] MAIO buffer in driver TX ring [d] MAIO buffers used by the device driver for RX

[1] In Linux, one the file descriptors has to be a pipe

3.1 Bifurcated I/O

To avoid the costly system calls, we delegate the I/O operation to a dedicated kernel thread (Fig. 1 **(3)**) which will perform the I/O operation using kernel sockets [25], this method has shown to be very efficient. For example a `send_msg` system call is replaced with an I/O descriptor (i.e., `struct msghdr` and `int flags`) written to a shared memory ring buffer (Fig. 1 **(2)**). This form of splitting the responsibility for performing I/O preservers the existing socket API, facilitates exception-less system calls, and allows for better parallel programming. Bifurcated I/O enables the separation of the application computations and the TCP computations to different CPU cores. Both zero-copy and standard send are supported; in standard mode, the sent buffer is copied to a new MAIO buffer before send. Supporting standard mode of send is beneficial for applications performing small I/O, e.g., 64B sends.

3.2 Security

Such a solution initially razes concerns about the system's security and stability, as the process now *seemingly* has access to sensitive kernel memory.

Kernel Isolation. MAIO is integrated in such a way that the shared pages are only ever used by the kernel to hold the I/O *data* buffers and *not* the metadata or any other kernel need. Namely, the process can only ever see the information it has written or data-bound to user-space. In addition to the data, the process is privy to the transport headers as well; we assume the NIC supports Header/Data splitting[10] which can place the headers onto non-shared buffers.

Driver Support. Modern NICs have the ability to create process dedicated rings (e.g., Intel ADQ [18], Mellanox NICs [28]). HW support[17] can direct a single 5 tuple (or a defined group of 5 tuples). Limiting the shared buffers *only* to this users data. The dedicated RX ring shown in Fig. 1 **(4)** is using MAIO pages (Fig. 1 **[d]**). In Fig. 1, we also see the kernel buffer (marked as the opaque box) being used by the same TX ring as a MAIO page(Fig. 1 **[c]**), no special care is needed for data separation. The implementation of a driver support, is out of the scope of this work.

User Isolation. Isolating data access between I/O streams of different processes is achieved with application dedicated rings, and each process has its instance of MAIO. For virtual environments, the hypervisor can trivially be extended to support application dedicated rings. On older systems with NICs that do not support dedicated rings, MAIO can still be used if usage is restricted only to a privileged user (e.g., root, netdev group).

3.3 Shared buffer concerns.

Kernel Starvation. The user process may hoard MAIO buffers without releasing them to the kernel. In this case, the driver will revert to standard memory allocation and render the application unable to receive, while other process and kernel functionality will remain intact.

Pinned pages. Zero-copy solutions with shared static buffer were once considered dangerous because these shared pages can be exhausted and cannot be swapped out [35, 40]. We contend that this is not a real concern for modern systems as systems with abundant memory are the norm. Key/Value applications (e.g., Memcached, Redis) expect their memory to be persistent in memory. Additionally HPC applications, many[39] of which use RDMA, `register` (i.e., pin to memory) large memory regions that are then used for I/O.

3.4 Zero Copy support for kernel sockets

We expand the existing Linux TCP API with a `tcp_read_sock_zcopy` for `RX` and add a new msg flag `SOCK_KERN_ZEROCOPY` for `tcp_sendmsg_locked` in `TX`.

RX. We base our new function `tcp_read_sock_zcopy` on existing infrastructure i.e., `tcp_read_sock`. It is used by `tcp_splice_read` to collect buffers from a socket without copying.

TX. Zero-copy infrastructure already exists in the form of MSG_ZEROCOPY[8]. When kernel memory is used for I/O, enabling zero-copy is trivial compared to zero-copy from userspace. The pages are already pinned in memory, and there is no need for a notification on `TX` completion.

`do_tcp_sendpages`. Instead of modifying the behavior of `tcp_sendmsg_locked`, its also possible to use `do_tcp_sendpages`, which is used in splice. Ironically, `do_tcp_sendpages` accepts only one page fragment (i.e., `struct page`, size and offset) per invocation and does not work with a scatter-gather list, which `tcp_sendmsg_locked` supports.

4 MAIO API

MAIO API is designed to provide a socket like interface with added support for async I/O operations.

Below is an example of a simple TCP client code.

```
#include <maio.h>

int main(void)
{
    struct iovec io_vec[16];
    /*Init MAIO pages*/
    void *cache    = init_hp_memory(PAGE_CNT);
    /* Create a TCP socket & connect */
    int    idx    = create_connected_socket(dip, port);
    /* get a 4K buffer from HP  */
    char *buffer  = alloc_page(cache);
    /* init the Async TX ring       */
    init_tcp_ring(idx, cache);
    /* Async Zero-Copy Send*/
    send_buffer(idx, buffer, len, flags);
    /* get page state */
    get_buffer_state(buffer);
    /* collect an sg list of RX buffers */
    tcp_receive_sg(idx, io_vec, 16, cache);
    return;
}
```

In our implementation, we used 2MB Huge Pages for MAIO memory. Thus the function `init_hp_memory` allocates `PAGE_CNT` 2MB huge pages and creates a memory translation table for this process. Allowing the kernel RX/TX flows to use and translate the addresses without accessing the users' page table. The `void *cache` is an opaque handler for custom user-space memory management (e.g, in DPDK this would be the `rte_mbuf_pool`).

Next `create_connected_socket` both creates a kernel TCP socket and connects to the destination. The return value `idx`

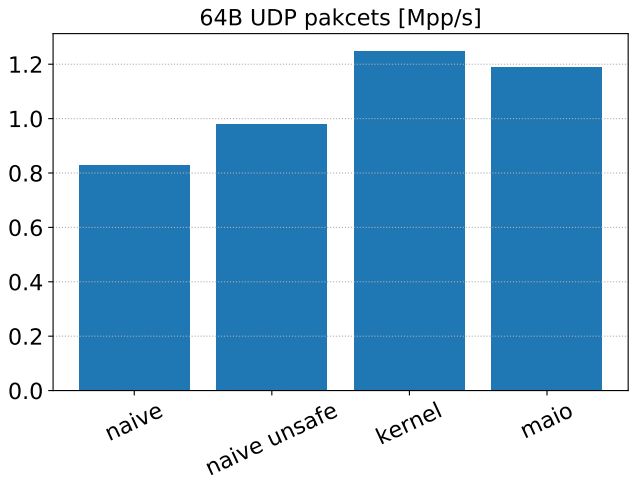


Figure 2: Number of 64B udp packets sent using user-space sockets(both with and without mitigations), kernel sockets and MAIO.

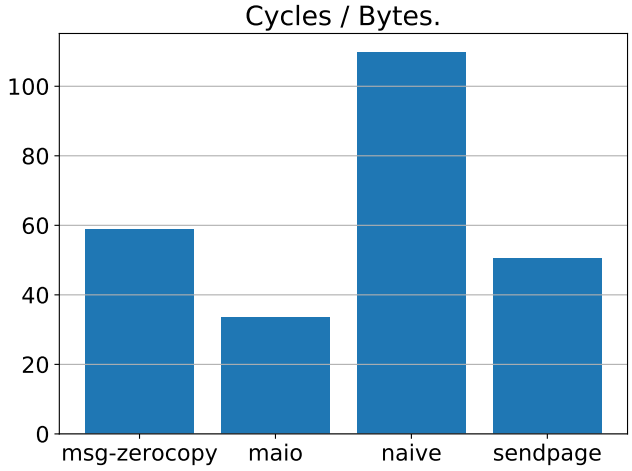


Figure 3: The average cycles per packet needed for each zero-copy method (16KB sends). [lower is better]

is an internal identifier used by MAIO rather than a standard file descriptor.

`init_tcp_ring` creates a ring for asynchronous I/O. This is similar to the function of `io_uirng`. The `cache` is used to provide memory to the shared ring, and `idx` is used to connect the ring with the kernel TCP socket.

`send_buffer` posts a buffer allocated by `cache` on the ring associated with the kernel socket and wakes the associated kernel thread that uses `tcp_sendpage` to send the page without copying.

`tcp_receive_sg` collects a scatter-gather list of buffers from the TCP socket. We are using a modified version of `skb_zerocopy_iter_stream` that is used by `tcp_mmap`. With MAIO we have no limitations on the pages other than being allocated from a MAIO page pool. The `cache` is provided in order to replenish the MAIO pool in the kernel used by the device drivers for RX.

5 EVALUATION

We evaluate the benefits of MAIO on a virtual environment in the Google Cloud Platform(GCP). For our testing, we have created three VMs; 16 core[2] Intel Cascade Lake high-throughput VMs capable of 32Gb/s of egress bandwidth[31], each with 64GB of RAM.

In all experiments, we have repeated the test 10 times, and we show the average result. We use perf[9] to analyze the major contributors to the cycles per byte costs of each of the tested techniques.

5.1 Bifurcated TX

In this experiment, we evaluate the direct benefit of bifurcated system calls. Each program sends a UDP stream of 64 Byte packets. The goal of this experiment is to gauge the impact of system calls on performance. We compare a simple `send_msg`, a kernel thread performing `kernel_send_msg` and MAIO. We run the naive version twice, once with `mitigations=off`[19] and once without this boot parameter. We present the number of sent datagrams per second in million packets per second(Mpp/s); the results are shown in Fig 2. We see that `mitigations`(naive unsafe in the figure) have a negative impact of 20% on the performance of the naive `send_msg`. Without mitigations, a kernel thread and MAIO still perform better than the naive thread by 25% and 20%, respectively.

[2]These VMs have two threads per physical core

5.2 Zero Copy TX

We evaluate the cost of data copying for a single sending process. To minimize the impact of system calls on performance, we send 16KB buffers. We evaluate a simple `send_msg`(i.e., naive), a `send_msg` with the `MSG_ZERO_COPY` flag, `sendfile` and MAIO. In this experiment, all but the `send_msg` sender were working at the rate of 27Gb/s. The `send_msg`, was bottlenecked on the core CPU and, as a result working at the rate of 19Gb/s. The experiment results are shown in Fig. 3, we show the Cycles/Byte metric as a meter of efficiency. We calculate the metric by measuring the CPU utilization in CPU cycles and dividing by the total bytes sent per second. In this experiment, about 50% of the CPU cycles for `send_msg` were spent on copying data; this amounts to about 55 CPU cycles spent on each sent Byte.With the `MSG_ZERO_COPY` flag, about 37% of the CPU cycles are spent on `get_user_pages_fast`, which translates to 22 cycles spent on dynamic remapping per byte. These results confirm the findings first reported in the original `MSG_ZERO_COPY` paper[8]. Sendpage, was closer in performance numbers to `MSG_ZERO_COPY` rather than to MAIO, due to the cost of `splice` abstractions, with 19% of the cycles spent on `generic_file_splice_read` and another 7% spent on call stack from `generic_splice_sendpage` to the actual `tcp_sendpage` that performs the TCP send. In the Linux kernel `send_msg` is implemented with `splice` abstractions.

5.3 Socket Splicing

We perform a simple TCP socket splicing experiment (i.e., Moving bytes from one TCP socket to another) akin to the TCP-split

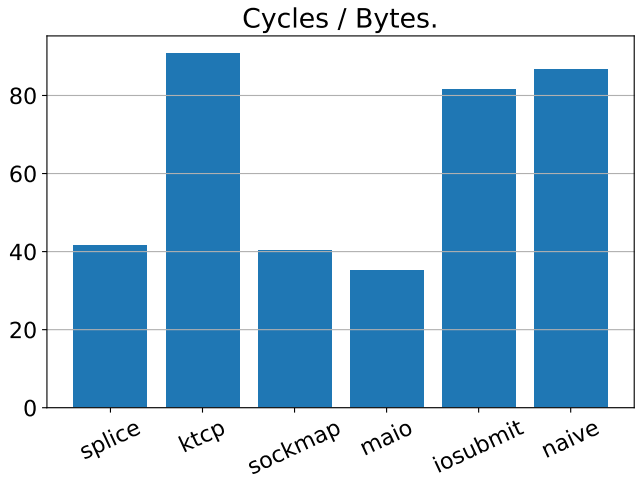


Figure 4: The average cycles per packet needed for each splicing solution. [lower is better]

functionality of KTCP. We measure each proposed solution's effectiveness by looking at the number of CPU cycles spent on average to splice (i.e., receive and send) a single byte. In this experiment we compare MAIO to user-space proxies implemented with splice[23], iosubmit[21] and BSD Sockets and also to KTCP[25] and SOCKMAP[14]. Results of the experiment are shown in Fig. 4.

We use 256KB buffers both for RX and TX, rendering the effect of system calls negligible. System calls account for less than 1% of the CPU syscalls of `iosubmit`,`splice` and naive. Unsurprisingly, KTCP, `iosubmit` and naive all spend between 25% to 32% of the cycles on memory copying. While both splice and sockmap perform much better than the copying functions, MAIO is still able to outperform both by about 10%. MAIO has none of the `splice` abstraction overheads between the actual TCP receive and send and none of the costs associated with eBPF.

6 FUTURE WORK

We plan on exploring the potential benefits of MAIO for a para-virtual system. We contend that a dedicated segment of host memory can be used for application dedicated para-virtual networking. Potentially allowing for an overhead free(e.g., copy, dynamic virtual memory remapping) zero-copy I/O from a guest OS's user-space, i.e., a MAIO page sent from a guest OS user program used directly by the physical NIC. MAIO would be able to facilitate more VMs/containers with fast networking on a single host which are also easier to migrate than is currently possible with SRIOV.

7 CONCLUSION

In this work, we have presented MAIO, a novel paradigm for user-space networking. MAIO facilitates zero-overhead network I/O, i.e., zero-copy and exception-less system calls. Unlike previous solutions, MAIO preservers the ubiquitous BSD Sockets API without sacrificing the system's performance or safety. MAIO is a much-needed step forward towards a performant and generic zero-copy technique.

REFERENCES

[1] Ethernet Alliance. Ethernet alliance roadmap. https://ethernetalliance.org/technology/2019-roadmap/, 2019.

[2] Jens Axboe. https://blogs.oracle.com/linux/an-introduction-to-the-io_uring-asynchronous-io-framework.

[3] Mark S Birrittella, Mark Debbage, Ram Huggahalli, James Kunz, Tom Lovett, Todd Rimmer, Keith D Underwood, and Robert C Zak. Intel® omni-path architecture: Enabling scalable, high performance fabrics. In *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*, pages 1–9. IEEE, 2015.

[4] Magnus Karlsson Björn Töpel. https://www.kernel.org/doc/html/latest/networking/af_xdp.html.

[5] Jeff Bonwick and Jonathan Adams. Magazines and vmem: Extending the slab allocator to many cpus and arbitrary resources. In *USENIX Annual Technical Conference, General Track*, pages 15–33, 2001.

[6] Shekhar Borkar, Robert Cohn, George Cox, Thomas Gross, HT Kung, Monica Lam, Margie Levine, Brian Moore, Wire Moore, Craig Peterson, et al. Supporting systolic and memory communication in iwarp. *ACM SIGARCH Computer Architecture News*, 18(2SI):70–81, 1990.

[7] Jonathan Corbet. https://lwn.net/Articles/776703/.

[8] Willem de Bruijn and Eric Dumazet. sendmsg copy avoidance with msg_zerocopy. *NetDevConf*, 2016.

[9] Arnaldo Carvalho De Melo. The new linux'perf'tools. In *Slides from Linux Kongress*, volume 18, pages 1–42, 2010.

[10] Dell. http://www1.la.dell.com/content/products/productdetails.aspx/switch-brocade-br1741m-k?c=dm&l=en&s=corp.

[11] DPDK. https://www.dpdk.org/.

[12] Ericc Dumazet. https://lwn.net/Articles/752188/.

[13] Alexander Duyck. https://www.kernel.org/doc/html/latest/vm/page_frags.html, 2016.

[14] John Fastabend. https://cilium.io/blog/2018/04/24/cilium-security-for-age-of-microservices/.

[15] Matt Fleming. https://lwn.net/Articles/740157/.

[16] Pål Halvorsen, Espen Jorde, K-A Skevik, Vera Goebel, and Thomas Plagemann. Performance tradeoffs for static allocation of zero-copy buffers. In *Proceedings. 28th Euromicro Conference*, pages 138–143. IEEE, 2002.

[17] Intel. https://software.intel.com/en-us/articles/setting-up-intel-ethernet-flow-director.

[18] Intel. Application dedicated queues(adq). https://netdevconf.info/0x14/session.html?talk-ADQ-for-system-level-network-io-performance-improvements.

[19] Linux. https://elixir.bootlin.com/linux/v5.0.21/source/Documentation/admin-guide/kernel-parameters.txt#L2542.

[20] Marek Majkowski. https://blog.cloudflare.com/sockmap-tcp-splicing-of-the-future/.

[21] Marek Majkowski. https://blog.cloudflare.com/io_submit-the-epoll-alternative-youve-never-heard-about/.

[22] Linux man-pages project. sendfile. http://man7.org/linux/man-pages/man2/sendfile.2.html.

[23] Linux man-pages project. splice. http://man7.org/linux/man-pages/man2/splice.2.html.

[24] Linux man-pages project. vmsplice. http://man7.org/linux/man-pages/man2/vmsplice.2.html.

[25] Alex Markuze, Bergman Aran, Dar Chen, Cidon Israel, and KEslassy Issac. Kernels of splitting tcp in the clouds. In *RADIO'19*, 2019.

[26] Alex Markuze, Adam Morrison, and Dan Tsafrir. True iommu protection from dma attacks: When copy is faster than zero copy. *ACM SIGARCH Computer Architecture News*, 44(2):249–262, 2016.

[27] Alex Markuze, Igor Smolyar, Adam Morrison, and Dan Tsafrir. Damn: Overhead-free iommu protection for networking. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 301–315, 2018.

[28] Mellanox/NVIDIA. Bifurcated driver. https://doc.dpdk.org/guides/howto/flow_bifurcation.html.

[29] OpenBSD. sosplice. https://man.openbsd.org/sosplice.9.

[30] Gregory F Pfister. An introduction to the infiniband architecture. *High performance mass storage and parallel I/O*, 42:617–632, 2001.

[31] Wells Philip and Jayadevan Manoj. https://cloud.google.com/blog/products/networking/google-cloud-networking-in-depth-how-andromeda-2-2-enables-high-throughput-vms.

[32] IOvisor Project. https://www.iovisor.org/technology/xdp.

[33] Luigi Rizzo. Netmap: a novel framework for fast packet i/o. In *21st USENIX Security Symposium (USENIX Security 12)*, pages 101–112, 2012.

[34] Livio Soares and Michael Stumm. Flexsc: Flexible system call scheduling with exception-less system calls. In *Osdi*, volume 10, pages 1–8, 2010.

[35] Jia Song and Jim Alves-Foss. Performance review of zero copy techniques. *International Journal of Computer Science and Security (IJCSS)*, 6(4):256, 2012.

[36] Alexei Starovoitov. https://lwn.net/Articles/599755/.

[37] Nico Struckmann, Yosandra Sandoval, Nadav Har'El, Fang Chen, Shiqing Fan, Justin Činkelj, Gregor Berginc, Peter Chronz, Niv Gilboa, Gabriel Scalosub, et al.

Mikelangelo: Micro kernel virtualization for high performance cloud and hpc systems. In *European Conference on Service-Oriented and Cloud Computing*, pages 175–180. Springer, 2017.

[38] Hari Subramoni, Ping Lai, Miao Luo, and Dhabaleswar K Panda. Rdma over ethernet—a preliminary study. In *2009 IEEE International Conference on Cluster Computing and Workshops*, pages 1–9. IEEE, 2009.

[39] TOP500. https://www.mellanox.com/solutions/high-performance-computing/top500.

[40] Shinichi Yamagiwa, Keiichi Aoki, and Koichi Wada. Active zero-copy: A performance study of non-deterministic messaging. In *The 4th International Symposium on Parallel and Distributed Computing (ISPDC'05)*, pages 325–332. IEEE, 2005.